Raphie Developer Challenge — Technical Documentation

Cold outreach email CLI · Node.js + OpenAI SDK + go-to-market helper · George Park · github.com/georgejpark/Raphie-DeveloperChallenge · October 25, 2026

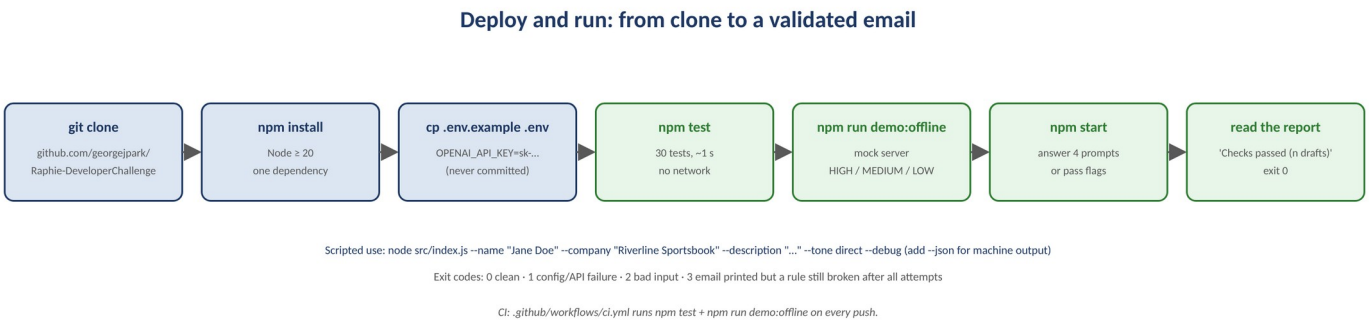
1 What this is and how the requirements are met

> A command-line tool that asks for a recipient, a company, a short description and a tone, enriches that with the go-to-market helper endpoint, drafts a cold outreach email from Raphie with the OpenAI API, and then proves the result follows Raphie's "must not feel AI-generated" rules before printing it. One runtime dependency (`openai`); everything else is Node core.

Requirement traceability

Brief requirement	Where it is implemented	How it is verified
1. Node.js application initialised with npm	`package.json` (ESM, `bin: raphie`, scripts `start` / `test` / `demo:offline`), `npm install` pulls the single dependency	`npm install && npm test` on Node 20/22; CI workflow in `.github/workflows/ci.yml`
2. CLI prompts for name, company, description, tone (readline)	`src/cli/prompts.js` — `node:readline` prompts with validation and re-ask; same values accepted as flags `--name --company --description --tone`	Interactive run and a piped-stdin run (lines are queued so piping works too); flag validation returns exit code 2
3. OpenAI SDK generates the email with the required content and tone	`src/email/generate.js` (`openai` package, `chat.completions.create`), `src/email/prompt.js` (writer's brief: name, company, commentary on their space, value proposition, call to action, tone guide)	`test/generate.test.js` with a fake client; live run with the key; `--debug` shows the brief that was sent
4. Helper endpoint integrated and used when it responds successfully; slow responses and 503s handled; low vs high confidence handled differently and documented	`src/gtm/client.js` (8 s timeout per attempt, 3 retries with backoff and jitter on 503/5xx/429/timeouts, 4xx not retried, malformed 200 rejected); `src/gtm/interpret.js` (HIGH ≥ 0.70 / MEDIUM 0.40–0.69 / LOW < 0.40 with the reasoning in the file header and in README §4)	`test/client.test.js` against a real local HTTP server (503 then success, budget exhausted, timeout, 4xx, malformed, connection refused); `test/interpret.test.js` for each tier
5. Helper response tailors the email beyond the description (operator vs B2B vs affiliate, their likely support challenges; judgment about which fields matter)	`src/gtm/interpret.js` playbooks per company type (audience, ticket shape, value proposition) + region notes + KYC/crypto/volume angles, capped at three; description always wins on conflict	`test/interpret.test.js` (description beats helper, affiliate ignores KYC, angle cap); `--debug` prints tier, company type and chosen angles
6. Output constraints: no em dash, no "Furthermore"/"Additionally" starting a sentence, no "fantastic", ≤ 1 "!", no two consecutive sentences with the same first word, ≤ 30 words per sentence	`src/email/validate.js` checks every draft; `src/email/generate.js` feeds violations back for a rewrite (max 3 drafts); `src/email/sanitize.js` repairs the mechanical rules as a last resort; exit code 3 if anything remains	`test/validate.test.js` (every rule, sentence splitting, the brief's two examples); `test/generate.test.js` (dirty → redraft → clean; three dirty drafts → sanitized)
Submission: code on GitHub, tag @agould65	Public repo `georgejpark/Raphie-DeveloperChallenge`, README with design notes, "with more time" section	—

2 Deploy and run — step by step



From clone to a validated email in seven steps.

#	Step	What happens / what you say
1	**Prerequisites**	Node.js 20 or newer (developed and tested on 22.22), npm, an OpenAI API key. No other system dependencies.
2	**Clone and install**	`git clone https://github.com/georgejpark/Raphie-DeveloperChallenge.git` → `cd Raphie-DeveloperChallenge` → `npm install` (installs `openai` only).
3	**Configure the key**	`cp .env.example .env` and set `OPENAI_API_KEY=sk-...`. The file is git-ignored; an exported environment variable works too and always wins over the file (`src/util/env.js`). Optional: `OPENAI_MODEL` (default `gpt-4o-mini`), `GTM_ENDPOINT`, `GTM_TIMEOUT_MS` (8000), `GTM_RETRIES` (3).
4	**Verify without network**	`npm test` runs 30 unit tests in about a second (no key, no network). `npm run demo:offline` starts a local mock of both external services and runs the full CLI three times: HIGH confidence, MEDIUM confidence with a 503 on the first helper call, LOW confidence.
5	**Run interactively**	`npm start` → answer the four prompts. Invalid tone or a description under three words is re-asked.
6	**Run scripted**	`node src/index.js --name "Jane Doe" --company "Riverline Sportsbook" --description "A sportsbook operator with high withdrawal volume, KYC questions, promo eligibility disputes, and periodic support spikes on big game days" --tone direct --debug`. Add `--json` for machine-readable output, `--skip-helper` to bypass the endpoint, `--max-attempts N` to change the redraft budget.
7	**Read the result**	The email, then one line: `Checks passed (n drafts)` or a warning listing any rule still broken. Exit codes: 0 clean, 1 configuration/API failure, 2 bad input, 3 email printed but a rule remains broken.

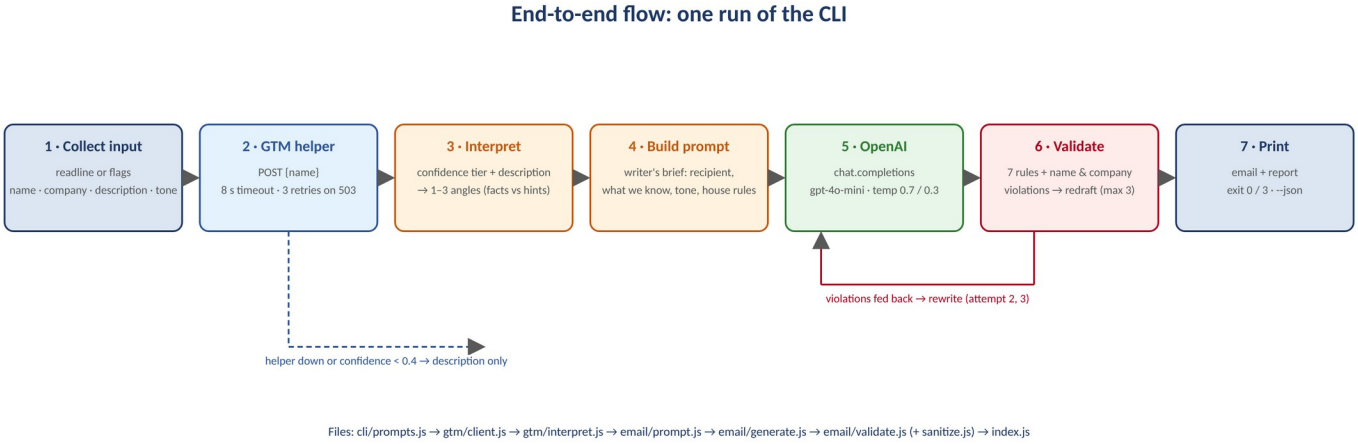
Sample session

```
$ npm start
Recipient's name (e.g. "Alex Gould"): Jane Doe
Company name (e.g. "Big Bet Sports"): Riverline Sportsbook
Short description of the company (e.g. "..."): A sportsbook operator with high withdrawal volume, KYC questions, promo eligibility disputes, and periodic support spikes on big game days
Tone (direct / warm) (e.g. "direct"): direct
Looking up go-to-market hints...
Drafting the email...

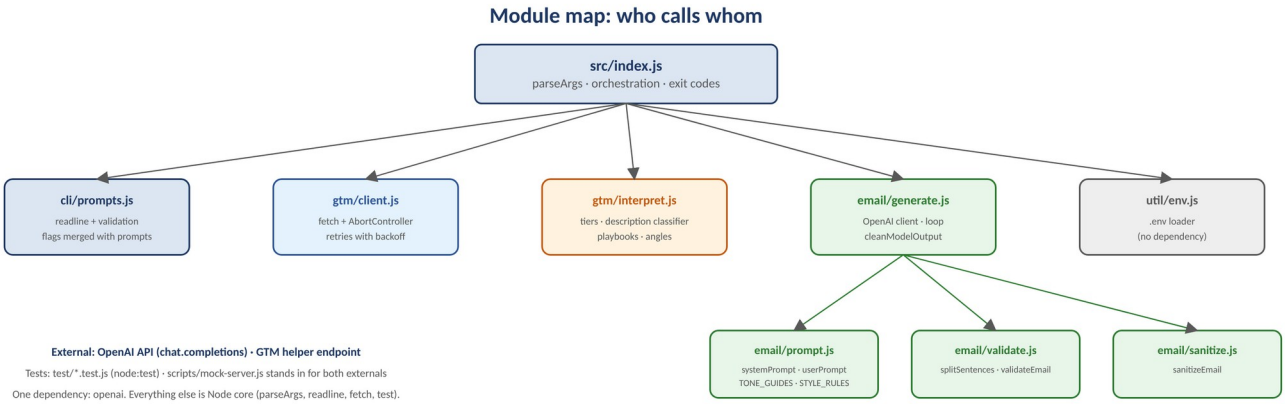
Hi Jane,
...
Best,
```

> Raphie
>
> Checks passed (1 draft).

3 Architecture and the end-to-end flow



One run of the CLI. The red loop is the redraft path; the dashed blue line is the fallback when the helper is unavailable or not trustworthy.



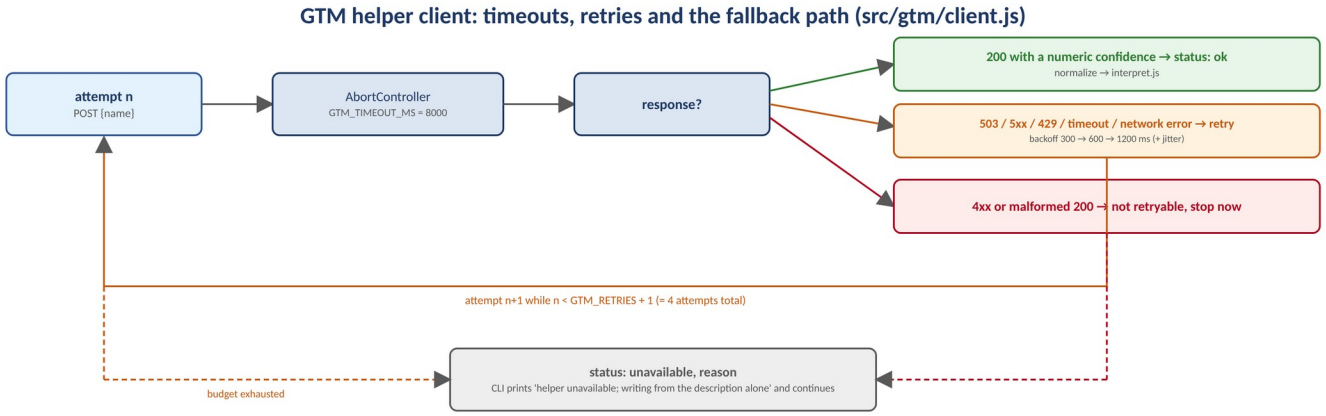
Module map. `index.js` orchestrates; every module is small, has one job, and is unit-tested in isolation.

Step by step, with code pointers

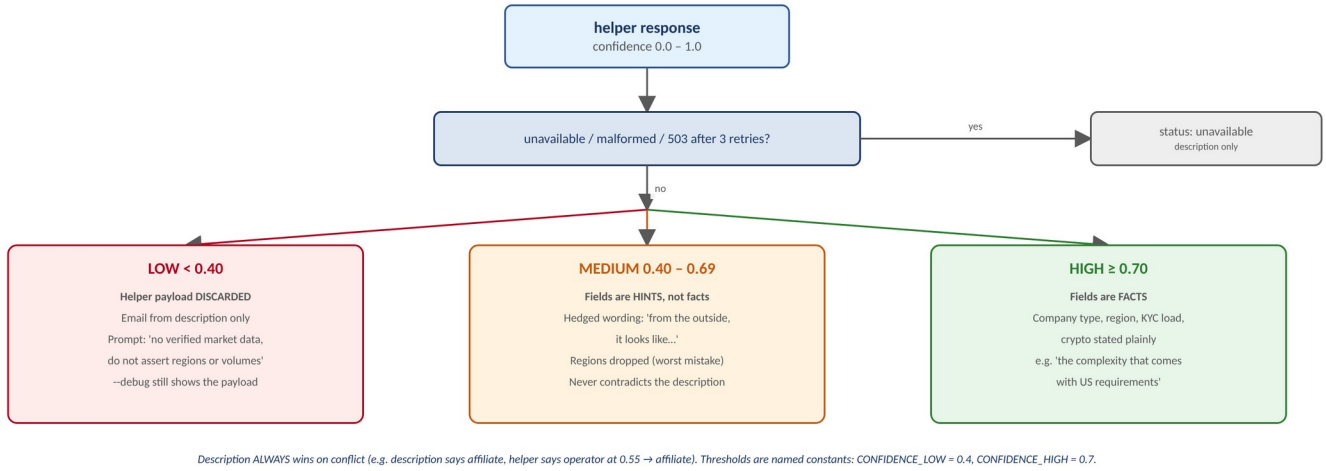
Step	What happens	Code
1 Collect	Flags are parsed with `node:util.parseArgs`. Anything missing is prompted with `readline`; each answer is validated and re-asked. The API key is checked before the questions so nobody types four answers and then hits a configuration error. Lines are queued so piped stdin works as well as a terminal.	`src/index.js` main(), `src/cli/prompts.js` collectInput(), lineReader()
2 Helper	`POST {name}` to the helper. Each attempt has an 8 s `AbortController` timeout. 503, 502, 504, 500, 429, 408, timeouts and network errors are retried with exponential backoff (300 → 600 → 1200 ms, plus jitter), up to 3 retries. 4xx is not retried; a 200 without a numeric `confidence` is treated as unavailable. Strings are lower-cased, regions upper-cased, confidence clamped to 0–1.	`src/gtm/client.js` fetchCompanyAnalysis(), normalize()
3 Interpret	The confidence is classified HIGH / MEDIUM / LOW. The description is read by a keyword classifier (company type, crypto, KYC, volume, regions, pain points). Both are merged with "description wins", producing at most three angles, each marked as a fact the writer may assert or a hint it may only suggest.	`src/gtm/interpret.js` confidenceTier(), analyzeDescription(), buildConditioning()
4 Prompt	A writer's brief: recipient; what we know and how sure we are (facts vs hints); what the email must do, in order; tone guide; house style. On a rewrite, the exact violations and the previous draft are appended.	`src/email/prompt.js` systemPrompt(), userPrompt(), TONE_GUIDES, STYLE_RULES

Step	What happens	Code
5 Generate	One `chat.completions.create` call per draft. Temperature 0.7 for the first draft, 0.3 for rewrites. Code fences, subject lines and curly quotes are stripped from the model output.	`src/email/generate.js` generateEmail(), cleanModelOutput()
6 Validate	Every draft is checked against the seven rules plus "first name and company present". Sentences are split on line breaks and on `.` `!` `?` + whitespace, with common abbreviations protected.	`src/email/validate.js` validateEmail(), splitSentences()
7 Redraft / sanitize	If a draft fails and attempts remain, the violations go back to the model. After the budget, the sanitizer fixes what can be fixed safely (em dash → comma, drop the banned starter, "fantastic" → "great", extra "!" → "."), and the result is validated one last time.	`src/email/generate.js`, `src/email/sanitize.js`
8 Output	Email + report, or `--json` with input, helper data, tier, angles, attempts and validation.	`src/index.js`

4 The go-to-market helper: reliability and the confidence policy



Timeouts, retries and the fallback. The helper is an enhancement, never a dependency.



Three tiers, three behaviours. The thresholds are named constants so they can be tuned once real precision data exists.

Why these thresholds

> A cold email that asserts a wrong fact about the recipient's own business is worse than one that says nothing: the reader knows their company and the error ends the conversation. So the question is not whether to use the helper data but how hard the email may lean on it. Below about 0.4 the helper is closer to guessing than knowing, so its payload is discarded and the email is written from the description alone (the prompt explicitly forbids asserting regions, licences or volumes). Between 0.4 and 0.7 the fields are hints: used only in hedged, outside-in language, never contradicting the description, and regions are dropped because a wrong region is the most embarrassing mistake to make. At 0.7 and above the fields are treated as facts the writer knows and may be stated plainly. `CONFIDENCE_LOW = 0.4` and `CONFIDENCE_HIGH = 0.7` live in `src/gtm/interpret.js` with this reasoning in the file header.

Which fields add value, and when they are noise

Field	Used for	Treated as noise when
companyType	Selects the playbook: audience, ticket shape and value proposition. Operator → containment of player questions (withdrawals, KYC, bonuses, account access, event spikes). Payments → two-sided demand (operators and end users), payout status, fraud reviews. B2B supplier → tier-1 triage, integration tickets, SLAs. Affiliate → agent assist over deflection, tracking and commission questions. Risk/compliance → case review and auditability. Data → technical triage. The single most valuable field.	The description already says what the company is; on conflict the description wins and the disagreement is logged with `--debug`.

Field	Used for	Treated as noise when
operatingRegions	One region note: US state-by-state requirements, UKGC affordability checks, LATAM payment rails and languages, and so on.	Confidence below HIGH; more than one region (only the first is used).
kycAmlIntensity	`high` adds the verification/KYC angle (repeat, time-sensitive tickets, auditability).	Company is an affiliate; value is `low` or `unknown`.
isCryptoRelated	Adds wallet, confirmation-time and AML questions.	`false`.
supportMotion	`high_volume` steers toward containment and spikes; `low_volume` toward agent assist and resolution time.	`mixed` / `unknown`.
vertical	Vocabulary (players vs customers).	Almost always; the playbook already carries it.
confidence	Gate for everything above.	Never.

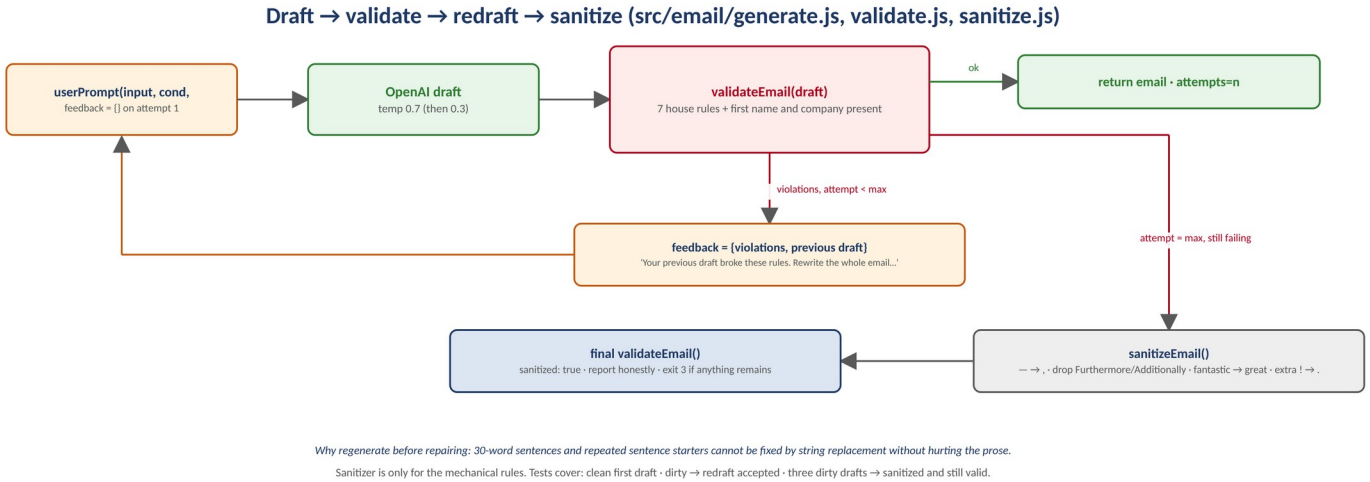
> At most three angles reach the prompt, in priority order: company type, crypto, one region, KYC, volume. A cold email that mentions everything says nothing.

5 Prompt design and dynamic conditioning

Part of the brief	Content	Conditioned on
System message	Who Raphie is and who it serves; write like an experienced sales engineer; output only the email body, no subject, no commentary.	Fixed.
Recipient	Full name and the first name to use, company name, the team's description verbatim.	CLI input.
What we know	"We know X is a B2C operator. Their audience is players." (HIGH or from the description) or "It appears X is…" (MEDIUM); pain points named in the description; a list of facts that may be stated plainly and a list of hints that must be hedged. With no usable helper data: "no verified market data, stay close to the description, do not assert regions or volumes".	Tier, company type source, angles.
What the email must do, in order	Greet by first name → say you are from Raphie and why → show you understand their world (the commentary on why their space is a strong candidate for automation) → state Raphie's value (the playbook's pitch, or the generic one) → call to action → sign-off.	Playbook, tone.
Tone guide	`direct`: get to the point, short sentences, numbers over adjectives, 15-minute call, "Best,"; `warm`: one human opener, collaborative, make it easy to say yes, "Warm regards,". Each has its own target length.	Tone flag.
House style	The seven rules in the model's language plus: no bullets, no subject line, do not invent numbers, do not mention AI. Repeated at the end because recency helps compliance.	Fixed.
Rewrite block	Only on attempt 2+: "Your previous draft broke these rules. Rewrite the whole email and fix every one:" followed by the exact violations and the previous draft.	Validator output.

> Run any input with `--debug` to see the tier, the company type and its source, and the angles (a trailing `?` marks a hedged hint). This is the fastest way to review how the conditioning behaves for a new kind of company.

6 Output constraints and enforcement



Regenerate first, repair last. The sanitizer only ever touches the mechanical rules.

Rule	Check in validate.js	Repair in sanitize.js (last resort)
No em dashes	— (U+2014) anywhere in the text	— → , (also en dash and - - , which are not banned but read as machine-written)
No "Furthermore" / "Additionally" starting a sentence	First word of each sentence, case-insensitive	Dropped; the next word is capitalised
No "fantastic"	Whole word, case-insensitive	→ "great"
At most one exclamation mark	Count of !	Second and later ! → .
No two consecutive sentences start with the same word	First words compared in reading order; greeting and sign-off lines count as units	Not repaired mechanically — regenerated with feedback
No sentence over 30 words	Words containing a letter or digit	Not repaired mechanically — regenerated with feedback
Name and company present (from the brief's content list)	First name; company or its first word ("Riverline" for "Riverline Sportsbook"), as in the brief's example	Regenerated with feedback

> A finding from writing the tests: both example emails in the brief contain sentences longer than 30 words (32 in example 1; 33 and 32 in example 2). The validator flags them; the tests assert that no other rule fires on the examples. The generator holds the stricter line, and the brief's examples are still the stylistic target.

7 Tests, offline demo and CI

Suite	What it proves	Technique
test/validate.test.js (11)	Every rule fires on a violating email and stays quiet on a clean one; sentence splitting respects line breaks and abbreviations; short company form accepted; the sanitizer repairs the mechanical rules without side effects; the brief's two examples pass everything except their long sentences.	Pure functions
test/interpret.test.js (8)	Tier boundaries (0.4 / 0.7 inclusive); the description classifier; HIGH asserts regions, MEDIUM hedges and drops regions, LOW discards the payload, "unavailable" behaves like LOW; description beats a contradicting helper; KYC is noise for an affiliate; at most three angles with company type first; the prompt text changes accordingly.	Pure functions + prompt inspection
test/client.test.js (7)	Request shape (`POST`, `{name}`); 503 twice then 200 → success on attempt 3; budget exhausted → unavailable with reason; 4xx not retried; slow response times out and the retry succeeds; malformed 200 not retried; connection refused → unavailable.	Real HTTP against a throwaway `node:http` server on 127.0.0.1
test/generate.test.js (4)	Clean first draft returned after one call; a dirty draft produces a second call whose prompt contains the violations and a lower temperature; three dirty drafts end sanitized and valid with the attempt history; model output cleaning.	Fake OpenAI client (records every request)
scripts/mock-server.js + demo-offline.js	The whole CLI, end to end, with no key and no network: HIGH tier, MEDIUM tier with a 503 on the first helper call, LOW tier; `MOCK_DIRTY_FIRST=1` makes the fake model break three rules on its first draft so the redraft is visible.	Local HTTP server implementing `/company-analysis` and `/v1/chat/completions`
.github/workflows/ci.yml	`npm ci`, `npm test`, `npm run demo:offline` on every push and pull request (Node 22).	GitHub Actions

Offline demo output (abridged)

```
==== HIGH confidence operator, direct tone, first draft deliberately dirty -> redraft ====
[debug] helper response: {"vertical":"igaming","companyType":"operator","operatingRegions":
["US"],"supportMotion":"high_volume","isCryptoRelated":false,"kycAmlIntensity":"high","confidence":0.85}
[debug] confidence tier: high; company type: operator (description); angles: type, region:US, kyc
[debug] attempt 1: 4 violation(s)
[debug] attempt 2: clean
Checks passed (2 drafts).

==== MEDIUM confidence payments company, warm tone, helper returns 503 once -> retry ====
[debug] helper attempt 1: HTTP 503
[debug] confidence tier: medium; company type: payments (description); angles: type, crypto, kyc?
Helper confidence is medium; its hints are used with hedged wording.
Checks passed (2 drafts).

==== LOW confidence -> helper discarded, description only ====
[debug] confidence tier: low; company type: affiliate (description); angles: type, assist
Helper confidence is low; its hints were not used.
Checks passed (2 drafts).
```

8 Design decisions, assumptions, and what I would do with more time

Decision	Reasoning
The helper is an enhancement, not a dependency	The tool must still produce a good email when the endpoint is slow, flaky or wrong, so the description-only path is a first-class path, not an error.
Description beats helper	The person typing the description has first-hand context; the helper fills gaps rather than overriding.
Fewer, sharper angles (max three)	Specificity is what makes a cold email feel human. Mentioning everything says nothing.
Validate, do not trust; regenerate before repairing	Prompt instructions are the first layer, the validator is the one that counts. Mechanical rewriting of long sentences produces worse prose than asking the model again with the exact violations.
Chat Completions API, gpt-4o-mini by default	Widest compatibility with SDK versions and OpenAI-compatible endpoints (which is also what makes the offline mock trivial); fast and cheap for 150-word emails. `OPENAI_MODEL` overrides.
No framework, one dependency	`parseArgs`, `readline`, `fetch`, `node:test` are all built in. Nothing to learn, nothing to patch.
Key handling	Read from the environment or a git-ignored `.env`; never written to the repository or to logs.

With more time

Idea	Why
Evaluation harness	Run 50–100 synthetic companies, store the drafts, score rule compliance, "AI smell" (second model as judge) and specificity; track first-draft pass rate per model and prompt version; tune 0.4/0.7 from data.
Structured output	Ask for `{greeting, paragraphs[], cta, signoff}` as JSON and assemble the email; structural checks become almost free and the sanitizer can work per sentence.
Few-shot by company type	One vetted example per playbook in the prompt.
Subject lines and variants	Two or three subject lines and an alternative opener, all through the same validator.
Market knowledge file	Regulator, payment rails and languages per market instead of one sentence each.
Caching, rate limits, observability	Cache helper responses by company for a day; honour `Retry-After`; log tier, angles, attempts and violations per run.
Packaging	npm bin, `--out file.txt`, CSV batch mode for a prospect list.